

Setup for push notification

ss2026

Setup before lab

- Ensure **Firestore, Storage, and Authentication** are already configured and fully functional in this project. *(No need to re-clone the project).*
- If you hasn't finished the setup teaching last weak, please follow the [link](#) to do the Firestore, Storage, and Authentication setup

Setup Functions (on your terminal)

- *\$ firebase init functions*

```
Functions can be deployed with firebase deploy.

✓What language would you like to use to write Cloud Functions? JavaScript
✓Do you want to use ESLint to catch probable bugs and enforce style? No
✓File functions/package.json already exists. Overwrite? No
i Skipping write of functions/package.json
✓File functions/index.js already exists. Overwrite? No
i Skipping write of functions/index.js
+ Wrote functions/.gitignore
✓Do you want to install dependencies with npm now? Yes
```

- *\$ flutterfire configure*

```
✓You have an existing 'firebase.json' file and possibly already configured your project for Firebase.
Would you prefer to reuse the values in your existing 'firebase.json' file to configure your project?
yes
```

- *\$ firebase deploy --only functions*

If you face the problem :
just try several time.

```
Error: There was an error deploying functions:
- Error Failed to create function groupChatAppSubscribeToTopic in region us-central1
- Error Failed to create function groupChatAppPushMessage in region us-west1
- Error Failed to create function groupChatAppSetModeratorCustomClaim in region us-west1
```

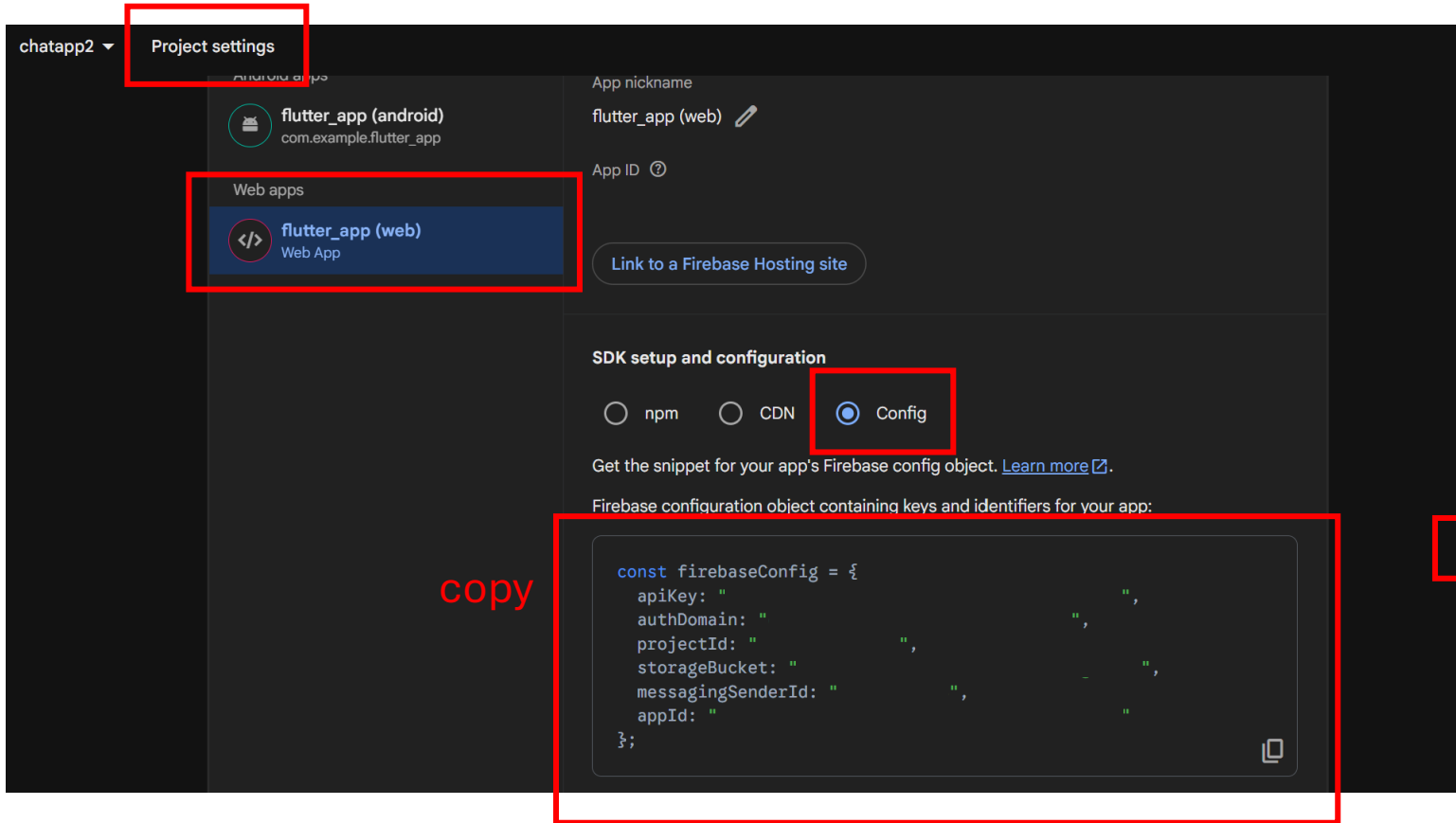
Setup for push notification

- **Web (follow the below pages)**
- For web users, please download web/firebase-messaging-sw.js and set up firebase.initializeApp. Go to Firebase Console > Your Project > Project Settings > Scroll down and click on your Web App, where you can find your apiKey, appId, and other configuration values.
- In push_messaging.dart set up your vapid key. You can find your vapid key in Firebase Console > Your Project > Project Settings > Cloud Messaging > Scroll to Web configuration > click Generate Key Pair > Copy the Key Pair
- make sure to enable notification permissions by going to: System Settings > Notifications > [Your browser, e.g., Chrome or Safari].
- **Android (follow the below pages) (we recommend to use web)**
 - **Step 1:** Add the SHA-256 fingerprint to your Firebase Android app.
 - **Step 2:** Refresh the console and download the updated google-services.json.
 - **Step 3:** Place the downloaded file into your local project's android/app/ directory.
- **iOS**
- For users running the iOS simulator, receiving notifications requires APNs, which in turn requires an Apple Developer account with proper permissions.
It is recommended to use the web simulator instead if you're unable to configure APNs.

Web setting - firebase-messaging-sw.js

- Verify if the file [web/firebase-messaging-sw.js](#) exists.
 - If it is missing, download it from the lab webpage and place it directly into the [web/firebase-messaging-sw.js](#).

Web setting - firebase-messaging-sw.js



Web setting - Vapid key

The screenshot shows the 'Project settings' page for a project named 'chatapp2'. The 'Cloud Messaging' tab is selected and highlighted with a red box. Below the tabs, the 'Web configuration' section is visible. Under 'Web configuration', the 'Web Push certificates' sub-section is active. It contains a description of Firebase Cloud Messaging using Application Identity key pairs, a 'Learn more' link, and a table with headers 'Key pair', 'Date added', 'Status', and 'Actions'. A 'Generate key pair' button is located below the table and is also highlighted with a red box.

chatapp2 ▾

Project settings

General **Cloud Messaging** Integrations Service accounts Data privacy Users and permissions Alerts

Web configuration

Web Push certificates

Firebase Cloud Messaging can use Application Identity key pairs to connect with external push services. [Learn more](#)

Key pair	Date added	Status	Actions
Generate key pair			

You can also import an existing key pair.

Web setting – Vapid key

Web configuration

Web Push certificates

Web Push certificates

Firebase Cloud Messaging can use Application Identity key pairs to connect with external push services. [Learn more](#)

Key pair	Date added	Status	Actions
B E	Jun 10, 2026		

```
push_messaging.dart 1, M X
lib > services > push_messaging.dart > _backgroundMessageHandler
1  import 'package:cloud_functions/cloud_functions.dart';
2  import 'package:firebase_messaging/firebase_messaging.dart';
3  import 'package:flutter/foundation.dart';
4  import 'package:flutter_app/repositories/user_repo.dart';
5  import "package:universal_html/html.dart" as html;
6
7  /// VAPID key for web push notifications
8  const String vapidKey = 'YOUR_VAPID_KEY_HERE';
```


Web setting – Cloud Run Access Permissions

- **Step 1: Find Your Functions Region**

Run the following command to check your deployed region on **google cloud shell**

\$ *gcloud functions list*

- **Step 2: Update Cloud Run Access Permissions**

- Allow unauthenticated invocations (allUsers) for both services.
- Note: Replace “your_region” with the region found in Step 1

```
$ gcloud run services add-iam-policy-binding groupchatappunsubscribefromtopic \
--member="allUsers" \
--role="roles/run.invoker" \
--region="your_region"
```

```
$ gcloud run services add-iam-policy-binding groupchatappsubscribetotopic \
--member="allUsers" \
--role="roles/run.invoker" \
--region="your_region"
```



```
mamiehsiao@cloudshell:~ (chatapp2-5641e)$ gcloud functions list
NAME: groupChatAppSubscribeToTopic
STATE: ACTIVE
TRIGGER: HTTP Trigger
REGION: us-central1
ENVIRONMENT: 2nd gen

NAME: groupChatAppUnsubscribeFromTopic
STATE: ACTIVE
TRIGGER: HTTP Trigger
REGION: us-central1
ENVIRONMENT: 2nd gen
```

step1

```
mamiehsiao@cloudshell:~ (chatapp2-5641e)$ gcloud run services add-iam-policy-binding groupchatappsubscribetotopic \
--member="allUsers" \
--role="roles/run.invoker" \
--region="us-central1"

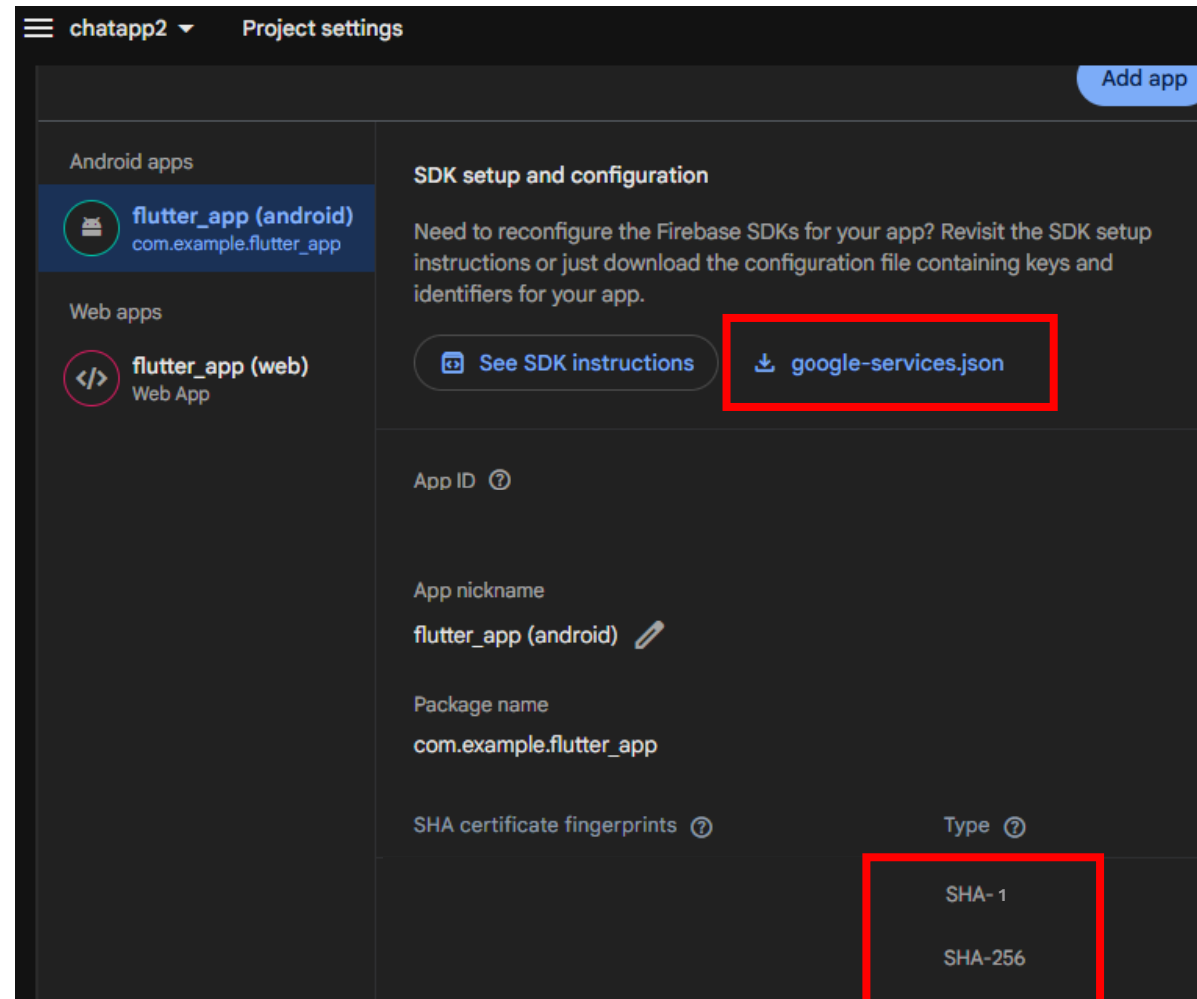
Updated IAM policy for service [groupchatappsubscribetotopic].
bindings:
- members:
  - allUsers
  role: roles/run.invoker
etag: BwZT4eue-FQ=
version: 1
```

step2

Android Emulator Setting

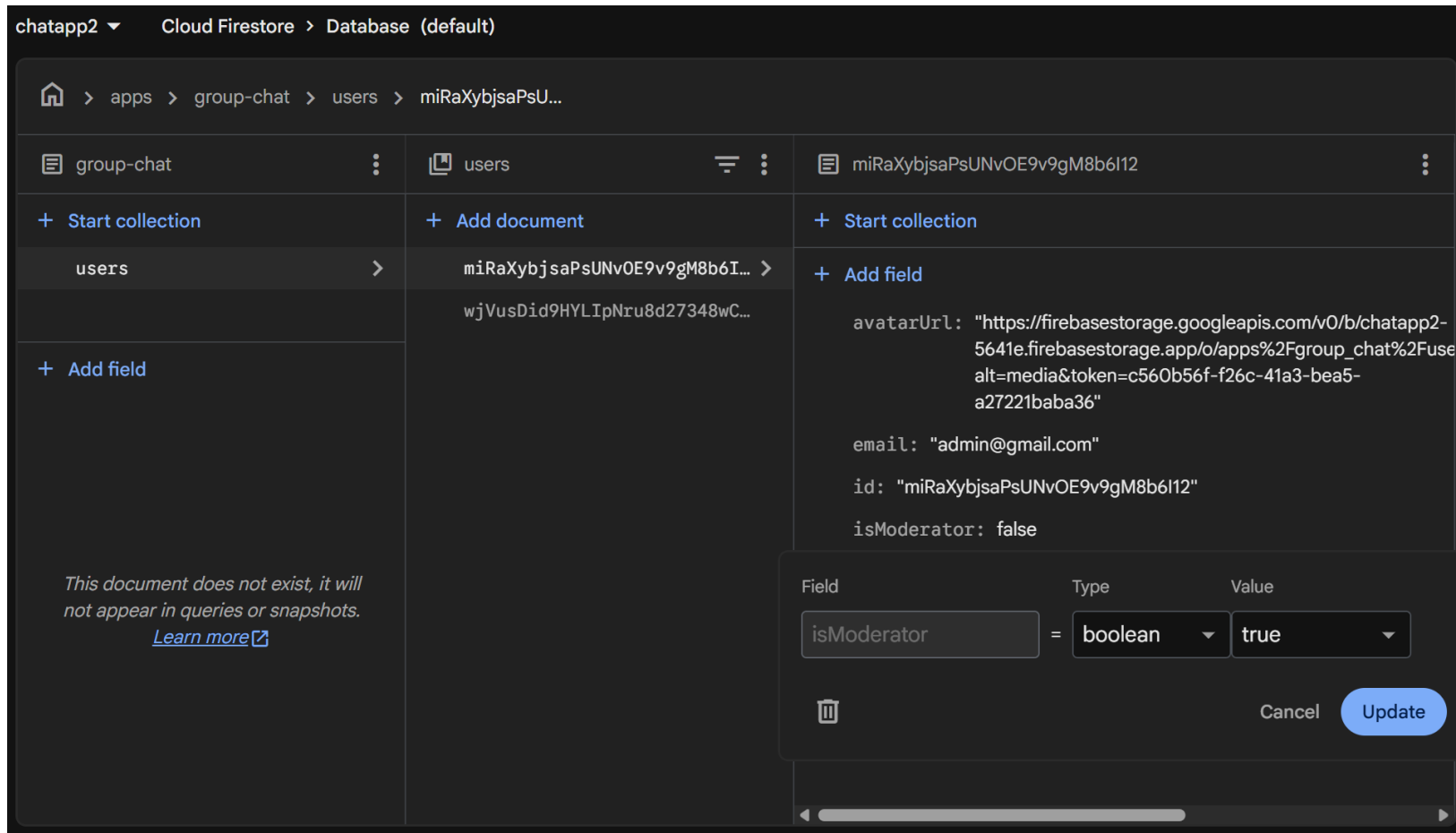
- In your cmd
\$ keytool -list -v -alias androiddebugkey -keystore "C:\Users\{your user name}\.android\debug.keystore"
Default Password: android
- Add the SHA-256 fingerprint to your Firebase Android app.
- Step 2: Refresh the console and download the updated google-services.json.
- Step 3: Place the downloaded file into your local project's android/app

Android Emulator Setting



Let a user become moderator

- Run your project and let a user become moderator



Run Your Project

- Test Notification Functionality:
 - Run the project simultaneously in two separate windows (e.g., normal and incognito mode).
\$ flutter run -d chrome --web-port=5000
\$ flutter run -d chrome --web-port=7000
 - Send a message from User A while User B is not on the active chat page. Verify that User B receives a browser notification.
 - Example setting finishing url: <https://youtu.be/2eZi1kilaIw>

Troubleshooting

- Troubleshooting (If notifications don't appear):
 - Ensure your computer is not on "Silent Mode" / "Do Not Disturb".
 - Check that Chrome permissions are not blocking notifications for this site.

